

CANCRO AL SENO: RAPPRESENTAZIONE, RICERCA, PREDIZIONE E PRENOTAZIONE

Componenti del gruppo

Alessandro Germinario[MAT.776901], [a.germinario20](#)

A.A. 2024-2025

[Link github](#)

Sommario

Capitolo 0) Introduzione	3
Capitolo 1) Creazione del dataset.....	4
Capitolo 2) Apprendimento non supervisionato	6
Capitolo 3) Apprendimento supervisionato	9
Capitolo 4) Classificazione con Naive Bayes e SVM.....	20
APPRENDIMENTO: CONCLUSIONI	24
Capitolo 5) Ragionamento logico e Prolog.....	24
Capitolo 6) Sistema di Prenotazione con CSP	26
Riferimenti bibliografici.....	28

Capitolo 0) Introduzione

L'obiettivo di questo progetto è sviluppare un sistema di classificazione in grado di predire la malignità di tumori al seno, basandosi su un dataset contenente diverse caratteristiche delle masse tumorali. L'output finale del progetto è un sistema che, dato un insieme di caratteristiche di un tumore, fornisce una predizione accurata della sua natura (benigna o maligna).

Requisiti funzionali:

- Linguaggio di programmazione: Python (Versione 3.12)
- IDE utilizzato: Visual Studio Code

Librerie utilizzate:

- **matplotlib**: per la visualizzazione dei grafici (curve di apprendimento, grafici a barre, ROC curve, Precision-Recall curve).
- **seaborn**: per la visualizzazione avanzata della matrice di confusione come heatmap.
- **numpy**: per la gestione e manipolazione degli array e delle matrici, operazioni numeriche.
- **pandas**: per l'importazione e gestione dei dataset in formato .csv, gestione dei dati tabulari.
- **scikit-learn**:
 - RandomForestClassifier: per la creazione e addestramento del modello di Random Forest.
 - DecisionTreeClassifier: per la creazione e addestramento del modello di Decision Tree.
 - LogisticRegression: per la creazione e addestramento del modello di regressione logistica.
 - cross_val_score: per la validazione incrociata (cross-validation) dei modelli.
 - train_test_split: per dividere i dati in set di addestramento e test.
 - StandardScaler: per la normalizzazione dei dati (scaling) in modo da migliorare le performance dei modelli.
 - roc_curve: per calcolare e tracciare la curva ROC.

- **roc_auc_score**: per calcolare l'AUC (Area Under the Curve) della curva ROC.
- **precision_recall_curve**: per calcolare e tracciare la curva Precision-Recall.
- **classification_report**: per generare il report di valutazione delle metriche di classificazione (precisione, recall, F1-score).
- **confusion_matrix**: per calcolare e visualizzare la matrice di confusione.
- **imblearn**: per gestire il bilanciamento delle classi, utilizzando tecniche come l'oversampling tramite SMOTE.
- **tqdm**: per la visualizzazione delle barre di avanzamento nei processi di training e validazione.
- **pyplot** (matplotlib): per la creazione di grafici (curves ROC, Precision-Recall, learning curve).

Installazione e avvio

Aprire il progetto con l'IDE preferito. Nel codice sono presenti alcune istruzioni commentate in quanto possono essere eseguite solo una volta (esempio: l'installazione dei requisiti). Avviare il programma partendo dal file `main.py`

Capitolo 1) Creazione del dataset

Il dataset utilizzato per il presente progetto è stato scaricato da Kaggle ed è relativo alla **classificazione del cancro al seno**. Il dataset contiene informazioni relative a diversi attributi biologici di tumori al seno e serve per predire se un tumore è maligno o benigno. Il file in formato CSV è stato importato e analizzato per eseguire un corretto preprocessing dei dati prima di essere utilizzato per il training dei modelli di machine learning.

Ecco una descrizione delle **features** utilizzate nel progetto:

1. **Features - Mean (Media)**: Le misurazioni medie delle caratteristiche del tumore, rappresentano la "tipica" misura di quella caratteristica per il tumore.
 - **radius_mean**: Il raggio medio del tumore.
 - **texture_mean**: La texture media del tumore, che indica la variabilità dei pixel.
 - **perimeter_mean**: Il perimetro medio del tumore.
 - **area_mean**: L'area media del tumore.
 - **smoothness_mean**: La lisciazza media del contorno del tumore.

- **compactness_mean**: La compattezza media del tumore, che rappresenta la sua forma.
- **concavity_mean**: La concavità media del tumore, che misura quanto il tumore presenta depressioni.
- **concave_points_mean**: Il numero medio di punti concavi nel contorno del tumore.
- **symmetry_mean**: La simmetria media del tumore.
- **fractal_dimension_mean**: La dimensione frattale media, che rappresenta la complessità del contorno del tumore.

2. **Features - SE (Deviazione Standard)**: Le misurazioni che indicano quanto ciascuna caratteristica del tumore varia, dando un'indicazione della sua irregolarità.

- **radius_se**: La deviazione standard del raggio del tumore.
- **texture_se**: La deviazione standard della texture del tumore.
- **perimeter_se**: La deviazione standard del perimetro del tumore.
- **area_se**: La deviazione standard dell'area del tumore.
- **smoothness_se**: La deviazione standard della liscezza del tumore.
- **compactness_se**: La deviazione standard della compattezza del tumore.
- **concavity_se**: La deviazione standard della concavità del tumore.
- **concave_points_se**: La deviazione standard del numero di punti concavi nel contorno del tumore.
- **symmetry_se**: La deviazione standard della simmetria del tumore.
- **fractal_dimension_se**: La deviazione standard della dimensione frattale del tumore.

3. **Features - Worst (Peggior)**: Le misurazioni che riflettono il valore peggiore o massimo osservato per ciascuna delle caratteristiche.

- **radius_worst**: Il raggio peggiore (massimo) del tumore.
- **texture_worst**: La texture peggiore del tumore.
- **perimeter_worst**: Il perimetro peggiore del tumore.
- **area_worst**: L'area peggiore del tumore.
- **smoothness_worst**: La liscezza peggiore del tumore.

- **compactness_worst:** La compattezza peggiore del tumore.
- **concavity_worst:** La concavità peggiore del tumore.
- **concave_points_worst:** Il numero peggiore di punti concavi nel contorno del tumore.
- **symmetry_worst:** La simmetria peggiore del tumore.
- **fractal_dimension_worst:** La dimensione frattale peggiore del tumore.

Preprocessing del dataset

1) **Importazione dei Dati:** Il dataset è stato importato da un file CSV, con la definizione dei tipi di dato per ogni colonna, per garantire che i dati vengano trattati correttamente (ad esempio, le variabili numeriche sono state trattate come float64 e la variabile target come object).

2) **Separazione delle Variabili:** È stata separata la variabile target (diagnosis) dalle variabili indipendenti (features) per il successivo processo di modellazione.

3) **Codifica della Variabile Target:** La colonna diagnosis, che contiene le etichette "B" (benigno) e "M" (maligno), è stata trasformata in numeri tramite il Label Encoding.

4) **Normalizzazione:** Sebbene non esplicitamente menzionato, una pratica comune è l'uso della standardizzazione delle feature numeriche per uniformare le scale dei dati, facilitando l'apprendimento del modello.

5) **Gestione dei Dati Sbilanciati:** usata la tecnica SMOTE per bilanciare il dataset in caso di classi sbilanciate (ad esempio, se ci sono più esempi di tumori benigni rispetto ai maligni).

6) **Divisione del Dataset:** Infine, il dataset è stato diviso in un set di addestramento e uno di test per addestrare il modello e valutarne le prestazioni.

Capitolo 2) Apprendimento non supervisionato

L'apprendimento non supervisionato è una branca dell'apprendimento automatico in cui l'agente viene addestrato su un insieme di dati senza etichette. Esistono due principali tipi di apprendimento non supervisionato:

- **Clustering** -> L'obiettivo è raggruppare gli elementi del dataset in base a delle somiglianze

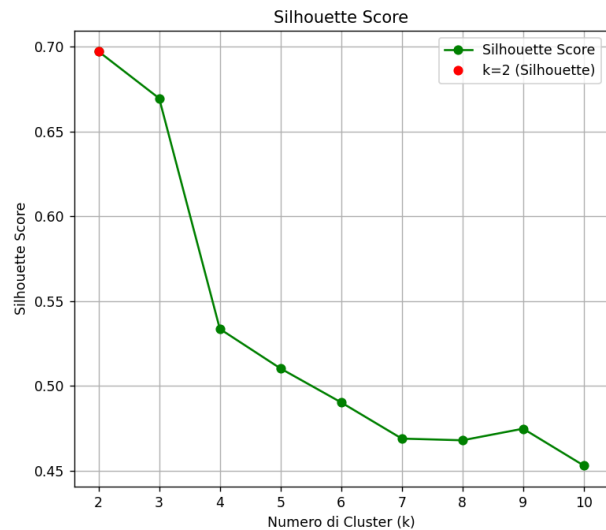
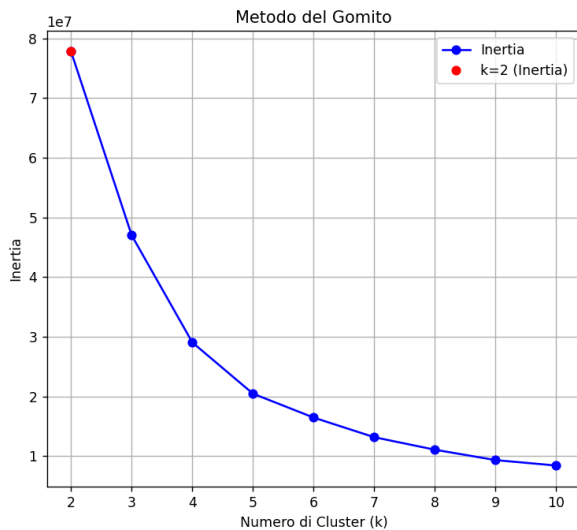
- Riduzione della dimensionalità -> L'obiettivo è ridurre il numero di feature utilizzate mantenendo però le informazioni più significative. Un esempio è la *PCA (Principal Component Analysis)*

Nel mio progetto, l'apprendimento non supervisionato è stato applicato con l'obiettivo di **raggruppare** i campioni del dataset in base alle caratteristiche numeriche, al fine di esplorare eventuali pattern nascosti nel cancro al seno. Questa tecnica di **clustering** può essere utile per individuare eventuali somiglianze tra i tumori maligni e benigni, o per identificare sottogruppi di dati che condividono caratteristiche comuni.

Per realizzare il clustering, ho utilizzato l'algoritmo di **KMeans**, che è una tecnica di **hard clustering**. Con KMeans, ogni esempio del dataset è assegnato a un cluster specifico, in base alla sua somiglianza con il centroide di un cluster. Un problema comune nel clustering con KMeans è scegliere il **numero di cluster** ottimale da fornire come input all'algoritmo.

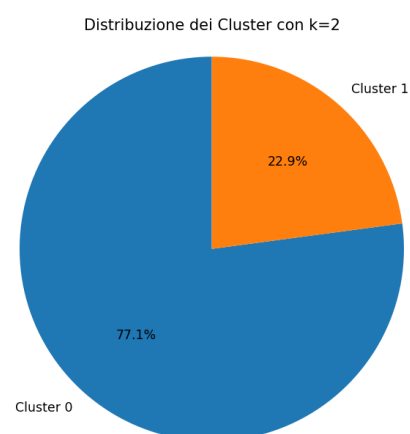
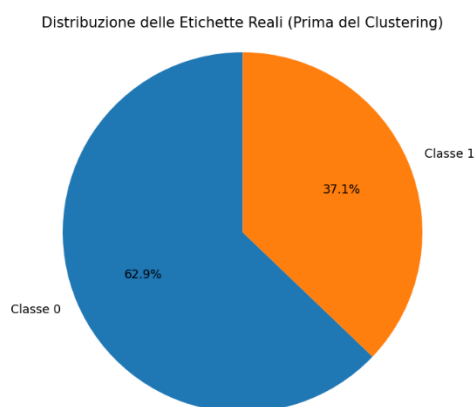
Nel mio caso ho deciso di utilizzare l'algoritmo di hard clustering *KMeans*. Uno dei problemi principali del *KMeans* è trovare il numero ideale di cluster da fornire in input all'algoritmo. Per risolvere questo problema ho seguito una strategia nota come "curva del gomito". La curva del gomito mostra la variazione dell'*inertia* al variare del numero di cluster, dove l'*inertia* rappresenta la somma delle distanze quadrate tra ogni punto dati e il centro del cluster assegnato. L'algoritmo prevede dunque di eseguire il *KMeans* per diversi valori di numero di cluster, calcolare l'*inertia* per ognuno di essi, plottare su un grafico la curva e poi identificare il gomito, ovvero il punto in cui la diminuzione dell'*inertia* diventa meno significativa. Nel mio progetto, oltre alla **curva del gomito** per determinare il numero ottimale di cluster, ho utilizzato anche due metodi di valutazione per **misurare la qualità del clustering**. **Silhouette Score**: Questa metrica misura quanto ogni punto sia ben raggruppato nel suo cluster rispetto agli altri cluster

Per quanto riguarda le scelte progettuali ho deciso di eseguire l'algoritmo con 10 valori di cluster, che variano da 1 a 10. Per quanto riguarda il parametro *n_init* questo indica le "RANDOM_RESTART". In particolar modo per ogni numero di cluster **i** verrà eseguito l'algoritmo per 5 volte e verrà restituito il clustering migliore. Per quanto riguarda il parametro *init='random'* significa che i centroidi vengono inizializzati in maniera del tutto casuale.



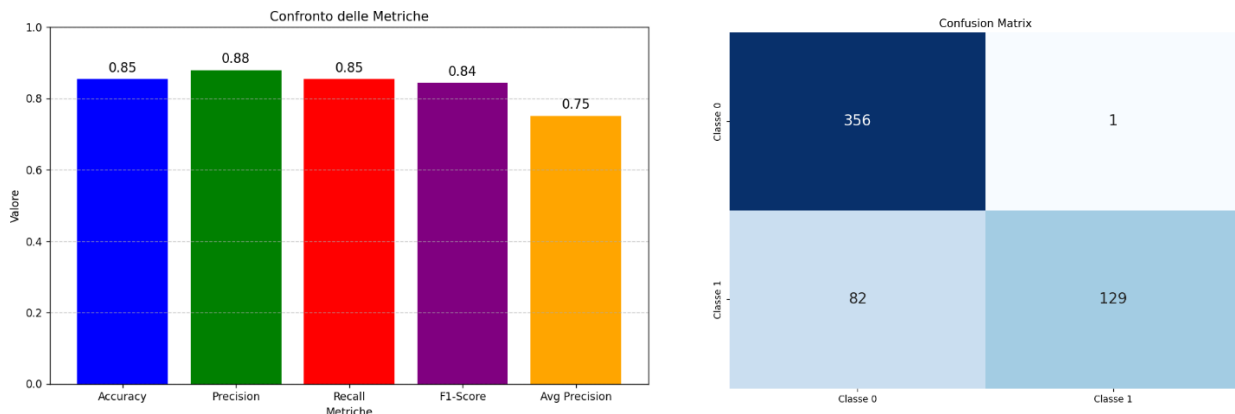
Nel contesto del mio progetto, l'analisi del numero ottimale di cluster è stata effettuata utilizzando la **curva del gomito**, che ha suggerito un valore di $k = 2$. Tuttavia, per una valutazione più accurata della qualità del clustering, è stata successivamente applicata la **Silhouette Score**, che ha identificato $k = 2$ come il numero ottimale di cluster. Questo ha portato all'esecuzione finale dell'algoritmo **KMeans** con **2 cluster**.

L'uso della **Silhouette Score** ha consentito di ottenere una valutazione più precisa della compattezza e separazione dei cluster, migliorando la validità del modello di clustering. Con il valore ottimale di $k = 2$, i cluster risultanti hanno mostrato una separazione significativa e una buona omogeneità interna, come evidenziato dai risultati finali dell'algoritmo.



Ogni cluster viene identificato da un indice.

Metriche e Matrice di Confusione per K-means



- Queste metriche aiutano a valutare l'efficacia del modello K-means nel classificare i dati in cluster corretti.
- La matrice di confusione fornisce una visione dettagliata delle previsioni del modello rispetto alle classi reali.

Capitolo 3) Apprendimento supervisionato

L'apprendimento supervisionato è una branca dell'apprendimento automatico in cui l'agente viene addestrato su un insieme di dati con etichette. Si divide in

- **Classificazione:** Le etichette rappresentano un dominio di valori discreto (quindi un insieme finito di valori) che la feature di output può assumere. Un caso particolare è la classificazione booleana, in cui le classi sono **true** e **false**.
- **Regressione:** L'etichetta può essere un qualsiasi valore numerico, quindi il dominio della feature di output è continuo.

L'obiettivo principale di questa parte del progetto è quello di costruire un modello di apprendimento supervisionato che, una volta addestrato, possa fare previsioni accurate su dati mai visti prima, imparando una relazione tra le variabili di input (features) e le etichette di output (diagnosi). Per raggiungere tale obiettivo, sono state affrontate le seguenti fasi:

1. **Fase di Addestramento:** Durante questa fase, il modello è stato addestrato sui dati di addestramento. L'algoritmo ha imparato la relazione tra le feature (caratteristiche del tumore) e la variabile target (diagnosi), cercando di ottimizzare la propria capacità predittiva.
2. **Fase di Test:** Una volta addestrato, il modello è stato testato su un insieme di dati mai visti prima, per valutarne l'efficacia nel fare previsioni accurate su nuovi dati. Questa fase è fondamentale per determinare la generalizzabilità del modello.

3. **Valutazione delle Prestazioni:** Le prestazioni del modello sono state valutate utilizzando metriche appropriate, come la **precisione**, il **richiamo**, la **f1-score**, la **matrice di confusione** e la **ROC curve**. Questo passaggio ha permesso di verificare quanto bene il modello fosse in grado di fare previsioni corrette per le due classi di diagnosi (benigno o maligno).

Per realizzare questa parte del progetto, sono state utilizzate diverse librerie Python:

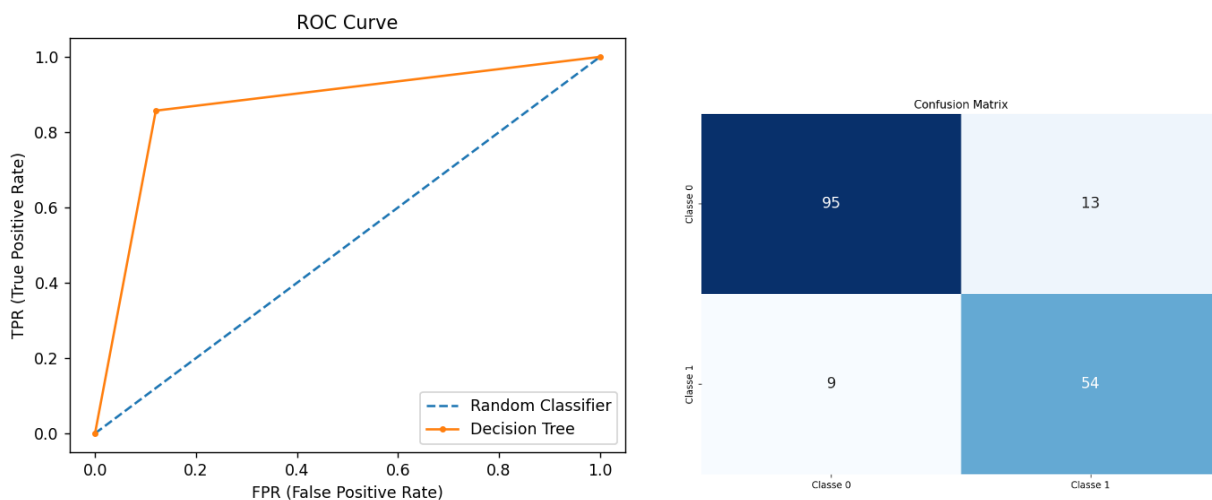
- **matplotlib** e **numpy** per la visualizzazione dei grafici e per il supporto nelle operazioni numeriche.
- **sklearn** per l'implementazione dell'algoritmo di apprendimento supervisionato, che include funzioni per l'addestramento e il test del modello.
- **pandas** per la gestione dei dati e la manipolazione del dataset.
- **imblearn.over_sampling** per affrontare il problema di classe sbilanciata, utilizzando tecniche come l'oversampling tramite il metodo SMOTE, al fine di migliorare la capacità del modello di classificare correttamente anche le classi meno rappresentate.

Questo processo ha consentito di costruire un modello robusto, in grado di fare previsioni accurate sulla diagnosi del cancro al seno, contribuendo così a una diagnosi precoce e più affidabile.

Nel mio caso l'apprendimento supervisionato è stato utilizzato con scopo di classificazione, dove la classi sono gli indici dei cluster prima ottenuti mediante apprendimento non supervisionato.

Per questo progetto ho deciso di utilizzare quattro modelli di apprendimento automatico

DecisionTree -> Il **Decision Tree** è un modello di apprendimento supervisionato che, per la classificazione, costruisce un albero in cui ogni nodo interno rappresenta una condizione su una feature del dataset, e ogni ramo rappresenta l'esito di quella condizione. Le **foglie** dell'albero rappresentano la classe di appartenenza dell'osservazione. Il modello segue il percorso nell'albero a partire dalla **radice** fino alla foglia corrispondente, basandosi sui valori delle feature di input.



La **ROC Curve** (Receiver Operating Characteristic) è uno strumento grafico utilizzato per valutare le prestazioni di un modello di classificazione. Mostra il **True Positive Rate (TPR)** rispetto al **False Positive Rate (FPR)** per diverse soglie di classificazione.

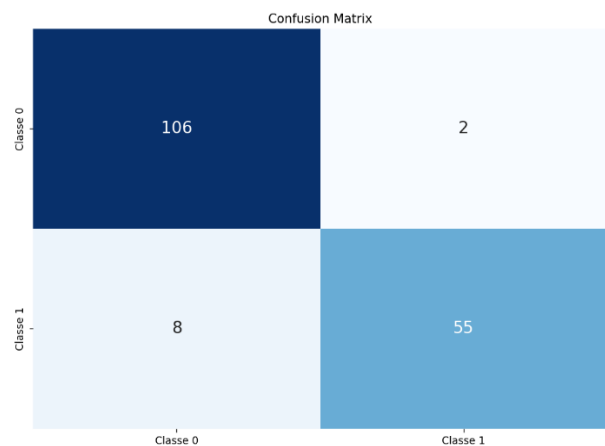
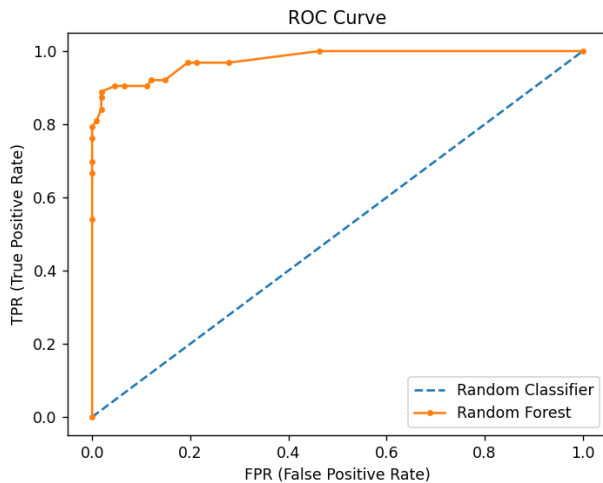
La curva ROC per il Decision Tree si avvicina all'angolo superiore sinistro del grafico, indicando buone prestazioni. Questo suggerisce che il modello è in grado di distinguere bene tra le classi.

Il valore AUC: 0.868 -> valore ottimale

La **matrice di confusione** fornisce una visione dettagliata delle previsioni del modello rispetto alle classi reali. Ecco i dati della matrice di confusione fornita:

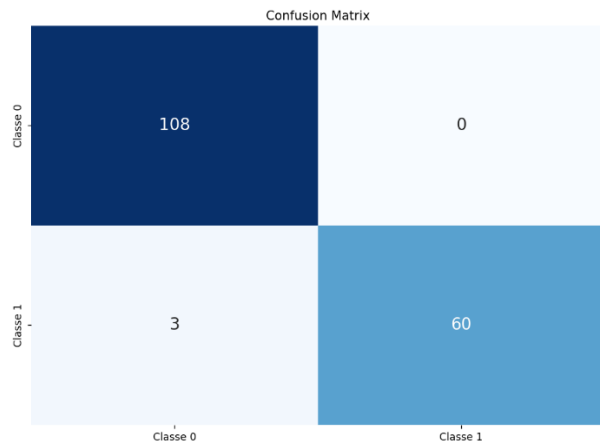
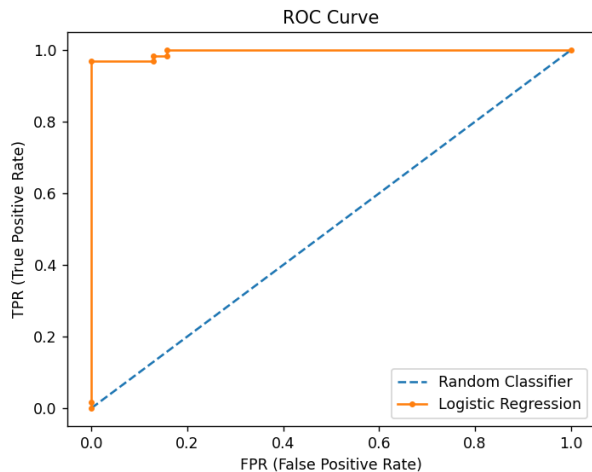
- Il modello ha una buona capacità di classificare correttamente la **Classe 0**, con solo 13 errori.
- Per la **Classe 1**, il modello commette meno errori (9 falsi positivi), il che suggerisce una buona accuratezza nella classificazione di questa classe.
- Il numero di **falsi negativi** per la Classe 0 è leggermente più alto rispetto ai falsi positivi per la Classe 1, il che potrebbe indicare una leggera tendenza del modello a classificare erroneamente alcune istanze della Classe 0 come Classe 1.

RandomForest -> Il **Random Forest** è un modello di ensemble che combina più **alberi decisionali** per migliorare la stabilità e l'accuratezza della previsione. Ogni albero nella foresta viene addestrato su un **sottoinsieme casuale** dei dati, utilizzando un campionamento con sostituzione (metodo **bagging**). La previsione finale è ottenuta mediando le previsioni di tutti gli alberi.



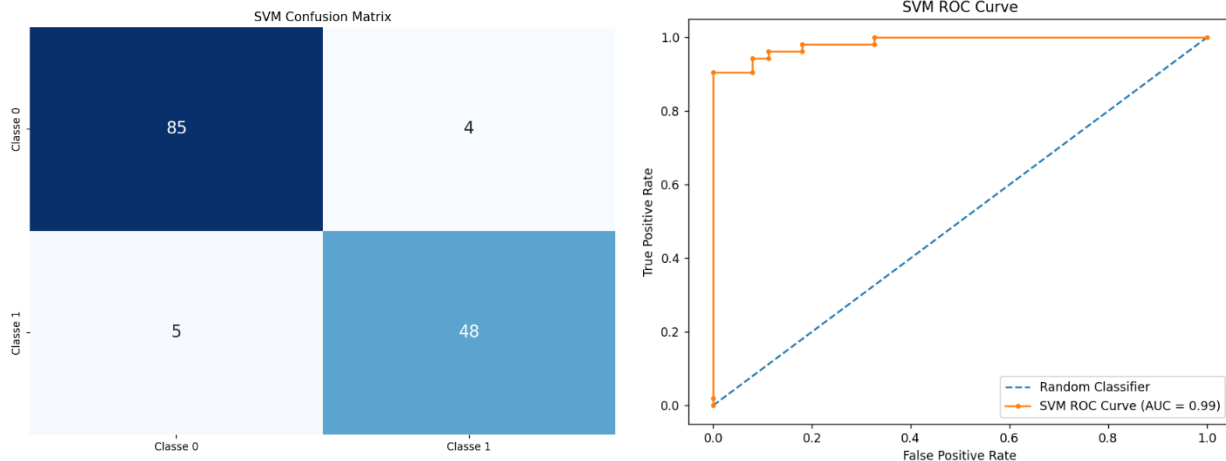
- **Random Forest:** La curva ROC per il Random Forest si avvicina all'angolo superiore sinistro del grafico, indicando ottime prestazioni. Questo suggerisce che il modello è in grado di distinguere molto bene tra le classi.
- **Random Classifier:** La linea diagonale rappresenta un classificatore casuale, che non ha alcun potere discriminativo. Il fatto che la curva del Random Forest sia ben al di sopra di questa linea indica che il modello è significativamente migliore di un classificatore casuale.
- **Area Under the Curve (AUC):** Anche se non esplicitamente calcolata, l'area sotto la curva ROC del Random Forest sembra essere molto ampia, indicando un alto livello di accuratezza nel distinguere tra le classi.
- Il modello ha un'ottima capacità di classificare correttamente la **Classe 0**, con solo 2 errori.
- Per la **Classe 1**, il modello commette un numero limitato di errori (8 falsi positivi), il che suggerisce una buona accuratezza nella classificazione di questa classe.
- Il numero di **falsi negativi** per la Classe 0 è molto basso, il che indica che il modello è molto efficace nel riconoscere correttamente le istanze della Classe 0.

Regressione Logistica Multinomiale (Funzione lineare) -> La **Regressione Logistica Multinomiale** è un'estensione della regressione logistica utilizzata per problemi di **classificazione multi-classe**. Aniché calcolare una probabilità per una sola classe (come nella regressione logistica binaria), il modello calcola una probabilità per ogni classe possibile, utilizzando funzioni lineari separate per ciascuna classe. Le probabilità vengono poi normalizzate utilizzando una funzione **softmax**.



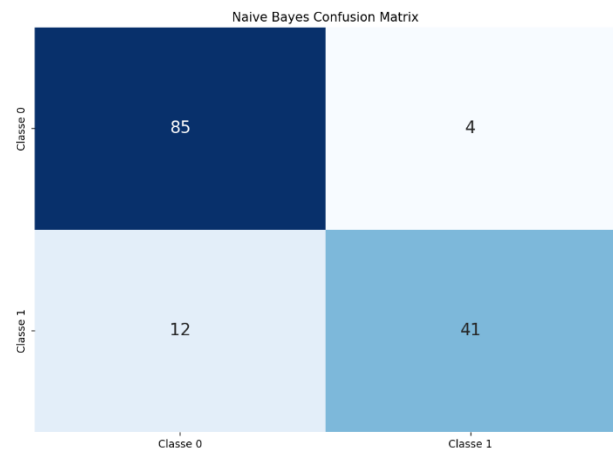
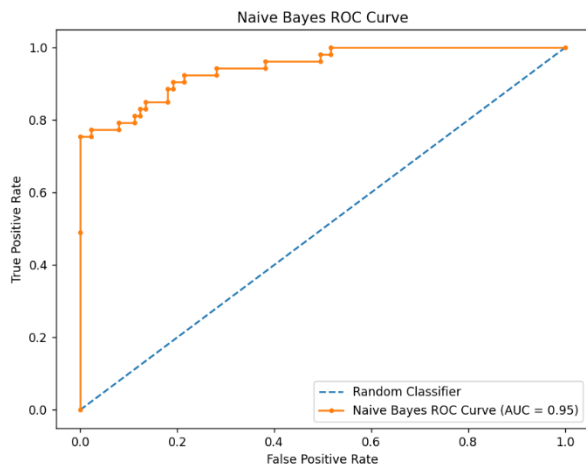
- **Logistic Regression:** La curva ROC per la Logistic Regression si avvicina all'angolo superiore sinistro del grafico, indicando buone prestazioni. Questo suggerisce che il modello è in grado di distinguere bene tra le classi.
- **Random Classifier:** La linea diagonale rappresenta un classificatore casuale, che non ha alcun potere discriminativo. Il fatto che la curva della Logistic Regression sia ben al di sopra di questa linea indica che il modello è significativamente migliore di un classificatore casuale.
- **Area Under the Curve (AUC):** Anche se non esplicitamente calcolata, l'area sotto la curva ROC della Logistic Regression sembra essere ampia, indicando un buon livello di accuratezza nel distinguere tra le classi.
- Il modello è particolarmente efficace nel classificare la **Classe 0**, senza commettere errori.
- Per la **Classe 1**, il modello commette un numero molto ridotto di errori, dimostrando una capacità di classificazione molto accurata.

Support Vector Machine (SVM)->La **Support Vector Machine (SVM)** è un modello di apprendimento supervisionato utilizzato principalmente per problemi di **classificazione**. SVM cerca l'iperpiano che separa le classi in uno spazio ad alta dimensione in modo da massimizzare il margine tra le classi. SVM è particolarmente potente quando le classi non sono linearmente separabili grazie all'uso del **kernel trick**, che mappa i dati in uno spazio di dimensioni superiori.



- **SVM:** La curva ROC per la SVM si avvicina molto all'angolo superiore sinistro del grafico, indicando prestazioni eccellenti. L'**Area Under the Curve (AUC)** è pari a **0.99**, il che suggerisce che il modello è quasi perfetto nel distinguere tra le classi.
- **Random Classifier:** La linea diagonale rappresenta un classificatore casuale, che non ha alcun potere discriminativo. Il fatto che la curva della SVM sia molto al di sopra di questa linea conferma che il modello è estremamente efficace.

Naive Bayes -> **Naive Bayes** è un modello di classificazione probabilistica che si basa sul **teorema di Bayes** e assume l'**indipendenza tra le feature**. Nonostante questa assunzione di indipendenza sia una semplificazione, il modello è sorprendentemente efficace in molte applicazioni.



- **Random Forest:** La curva ROC per il Random Forest si avvicina molto all'angolo superiore sinistro del grafico, indicando prestazioni eccellenti. Questo suggerisce che il modello è in grado di distinguere molto bene tra le classi.
- **Random Classifier:** La linea diagonale rappresenta un classificatore casuale, che non ha alcun potere discriminativo. Il fatto che la curva del Random Forest sia molto al di sopra di questa linea conferma che il modello è estremamente efficace.
- Il modello ha un'ottima capacità di classificare correttamente la **Classe 0**, con solo 2 errori.
- Per la **Classe 1**, il modello commette un numero limitato di errori (8 falsi positivi), il che suggerisce una buona accuratezza nella classificazione di questa classe.
- Il numero di **falsi negativi** per la Classe 0 è molto basso, il che indica che il modello è molto efficace nel riconoscere correttamente le istanze della Classe 0.

Fase di addestramento e test

Suddivisione del Dataset: Il dataset è stato suddiviso in due insiemi distinti:

- **Training set:** una porzione dei dati è stata utilizzata per addestrare i modelli.
- **Test set:** l'altra porzione dei dati è stata utilizzata per testare le prestazioni dei modelli.

La suddivisione è stata effettuata usando la funzione `train_test_split` della libreria `sklearn.model_selection`.

```
# Suddivisione del dataset in train e test (75% train, 25% test)
x1_train, x1_test, y1_train, y1_test = train_test_split(x1, y1, train_size=0.75, random_state=13)
```

Fase 3) Valutazione delle prestazioni

Sono state utilizzate le seguenti metriche per comparare le performance dei modelli di classificazione (DecisionTree, RandomForest, Regression Logistic)

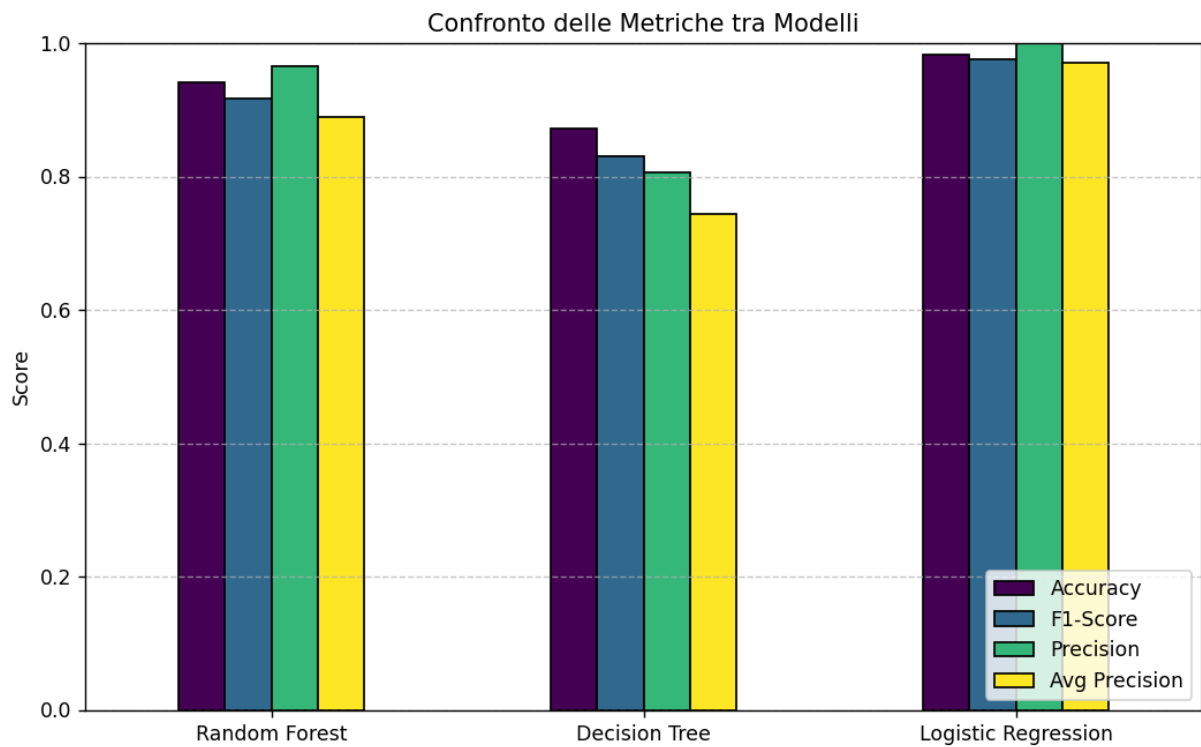
Accuracy (Accuratezza)->L'accuracy rappresenta la frazione di previsioni corrette rispetto al numero totale di previsioni. È una misura generale della capacità del modello di fare previsioni corrette, ma può risultare fuorviante in presenza di dataset sbilanciati.

Precision (Precisione)-> La precision è calcolata per ciascuna classe. Indica la capacità del modello di non classificare erroneamente esempi in classi non corrette (false positive).

F1-Score->L'F1-Score è la media armonica tra precisione e recall, combinando entrambi gli aspetti. L'F1-Score macro calcola la media di questi punteggi per ciascuna classe. È una metrica utile quando si desidera bilanciare precisione e recall, e viene spesso utilizzata quando le classi sono sbilanciate.

AVG Precision -> L'Average Precision (Avg Precision) è una metrica che sintetizza la **precisione media** del modello su tutte le soglie di probabilità. È utile quando si desidera misurare la qualità del modello in presenza di classificazione binaria o multi-classe, con una particolare attenzione a come il modello gestisce i trade-off tra precisione e recall su diverse soglie di decisione. La precisione viene calcolata per ogni punto di decisione e la media di queste precisioni fornisce una valutazione complessiva.

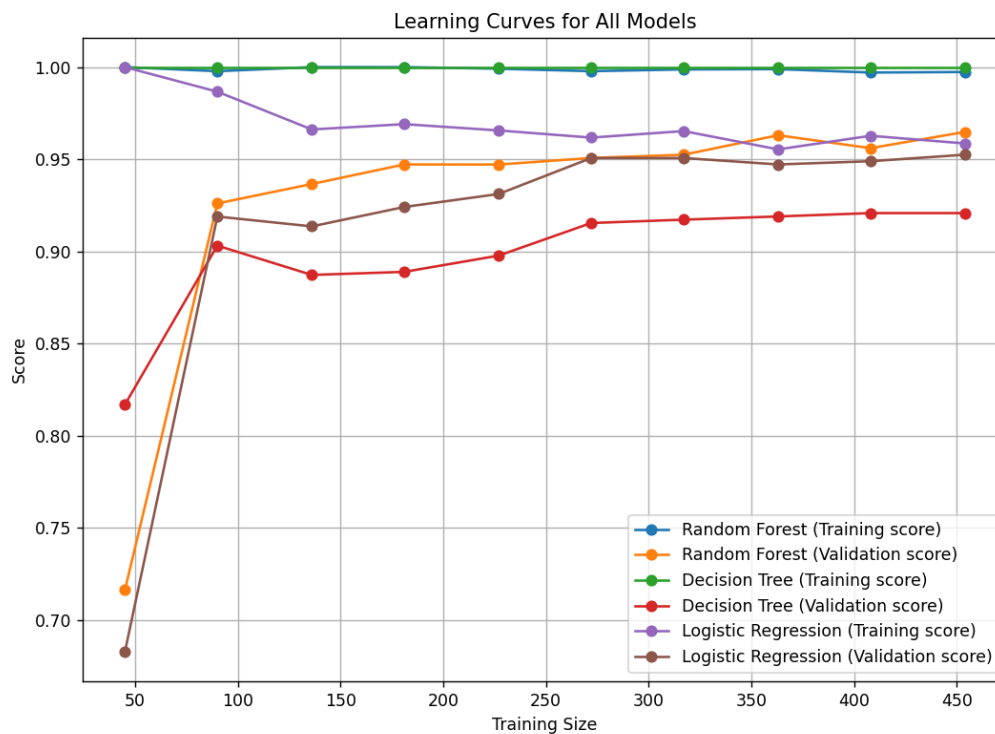
RISULTATI



Al termine dell'addestramento dei modelli, sono stati calcolati i valori delle metriche per ciascuno di essi e confrontati. I risultati forniscono una valutazione globale delle prestazioni dei modelli e aiutano a determinare quale di essi offre il miglior compromesso tra le diverse esigenze del progetto.

Possiamo notare che in generale i valori sono molto buoni per ogni metrica in ogni modello.

Curve di apprendimento confrontate



Analisi dei risultati per ogni modello

L'analisi delle learning curves (curve di apprendimento) è fondamentale per valutare le prestazioni dei modelli durante l'addestramento e per identificare eventuali problemi come overfitting o underfitting.

Random Forest

- **Training Score:** La curva del training score inizia alta e rimane stabile all'aumentare della dimensione del training set. Questo indica che il modello è in grado di adattarsi molto bene ai dati di training.
- **Validation Score:** La curva del validation score inizia più bassa ma aumenta rapidamente, avvicinandosi al training score man mano che il training set cresce. Questo suggerisce che il modello generalizza bene sui dati non visti.

Decision Tree

- **Training Score:** La curva del training score è molto alta sin dall'inizio e rimane costante, indicando che il modello si adatta perfettamente ai dati di training.
- **Validation Score:** La curva del validation score inizia bassa e aumenta lentamente, ma rimane significativamente più bassa rispetto al training score, anche con un training set più grande.

Logistic Regression

- **Training Score:** La curva del training score inizia bassa e aumenta gradualmente con l'aumentare della dimensione del training set.

- **Validation Score:** La curva del validation score segue un andamento simile a quella del training score, ma rimane leggermente più bassa.

Confronto Generale dei Modelli

Modello	Training Score	Validation Score	Overfitting	Underfitting	Adatto al problema?
Random Forest	Alto e stabile	Alto e convergente	No	No	Sì
Decision Tree	Molto alto	Basso	Sì	No	No (senza regolarizzazione)
Logistic Regression	Basso e crescente	Basso e crescente	No	Sì	No (troppo semplice)

POSSIBILE SOLUZIONE

Overfitting si verifica quando un modello è troppo complesso e si adatta eccessivamente ai dati di training, perdendo la capacità di generalizzare a nuovi dati. Nel caso del Decision Tree, l'alto training score e il basso validation score indicano che il modello ha memorizzato troppo i dettagli dei dati di addestramento.

Limitarne la profondità (max_depth):

- Impostare un valore massimo per la profondità dell'albero (`max_depth`) limita il numero di decisioni che possono essere fatte, evitando che il modello diventi troppo complesso e si adatti troppo ai dati di training.

Aumentare il parametro `min_samples_split` e `min_samples_leaf`:

- **`min_samples_split`:** Numero minimo di campioni richiesti per dividere un nodo interno. Un valore più alto impedisce che il modello si adatti troppo ai dati.
- **`min_samples_leaf`:** Numero minimo di campioni richiesti per essere un nodo foglia. Anche qui, valori più alti aiutano a generalizzare.

Underfitting si verifica quando il modello è troppo semplice per catturare la complessità dei dati. Nel caso della **Logistic Regression**, la curva di training score aumenta lentamente e rimane più bassa rispetto a quella di validation, suggerendo che il modello non sta imparando abbastanza dai dati di training.

Modificare i parametri di regolarizzazione (C):

- La **Logistic Regression** include un parametro di regolarizzazione, **C**, che controlla l'intensità della regolarizzazione. Un valore troppo elevato di **C** può rendere il modello troppo semplice, mentre un valore troppo basso può causare overfitting. Prova a esplorare diverse impostazioni di **C** per migliorare le prestazioni.

Utilizzare un modello non lineare:

- Se la relazione tra le feature e la target variabile è complessa e non lineare, puoi considerare modelli più complessi come **Support Vector Machines (SVM)** o **Random Forests**.

Capitolo 4) Classificazione con Naive Bayes e SVM

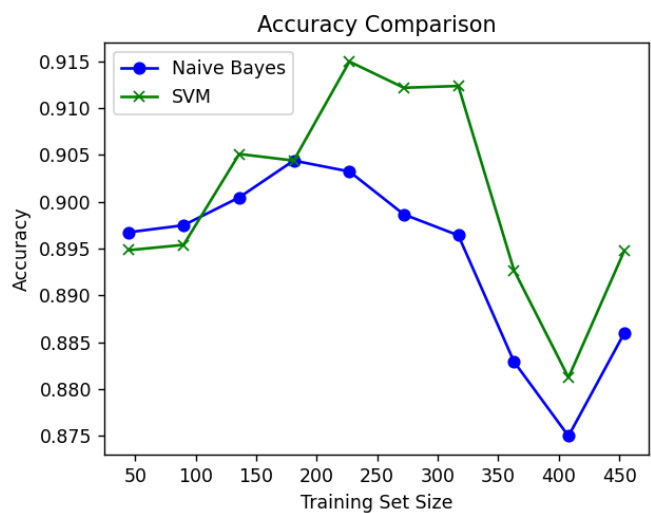
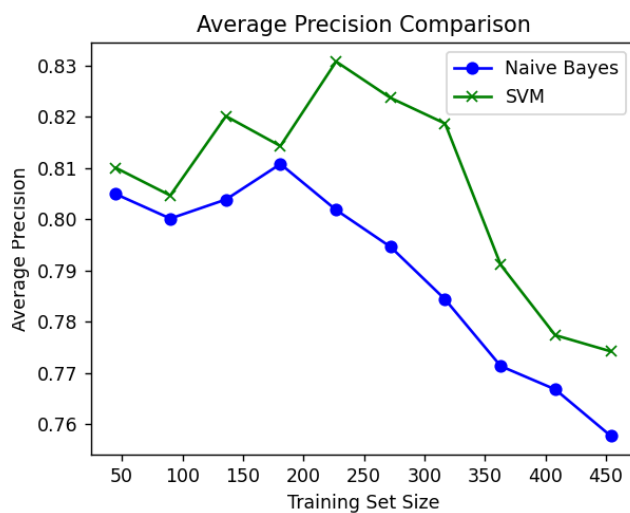
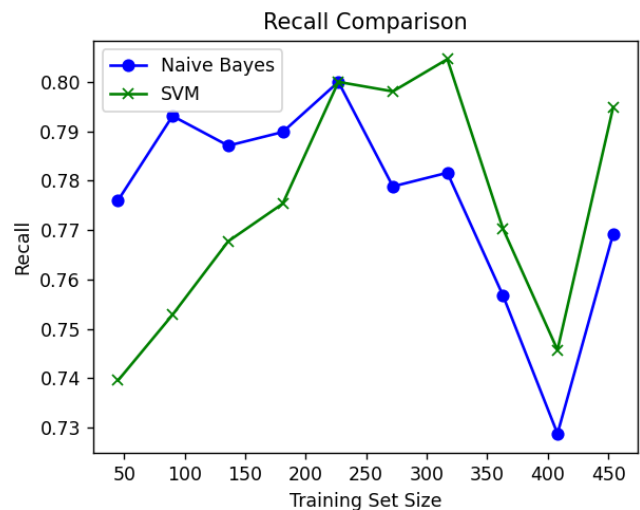
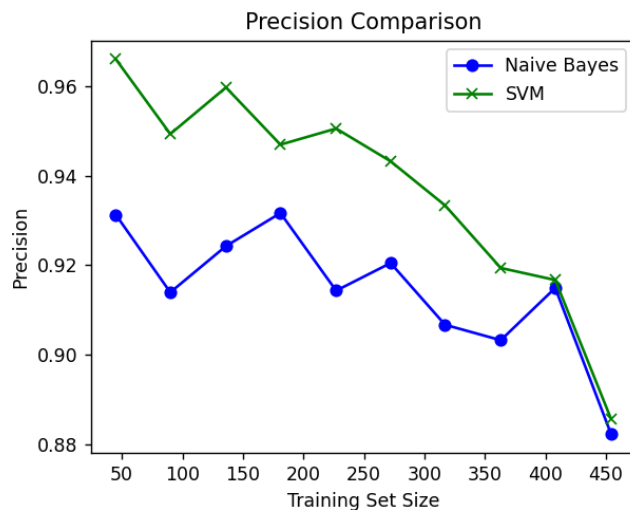
Nella data science e nel machine learning, la scelta del modello giusto è cruciale per ottenere prestazioni ottimali su un dato problema. In questo contesto, i modelli **Naive Bayes** e **Support Vector Machine (SVM)** sono due approcci comuni per la classificazione. Ognuno ha caratteristiche specifiche che lo rendono più o meno adatto a seconda del tipo di dati e del problema. La presente documentazione fornirà un confronto tra i due modelli in termini di performance, visualizzazioni e comportamenti sui dati.

-Il **Naive Bayes** è un modello probabilistico basato sul teorema di Bayes con l'assunzione di indipendenza tra le caratteristiche. Questo modello è particolarmente utile per problemi di classificazione dove le variabili predittive sono condizionate in modo indipendente.

-La **Support Vector Machine (SVM)** è un potente classificatore che cerca di trovare l'iperpiano ottimale che separa le classi nel miglior modo possibile, massimizzando il margine tra i punti di dati di diverse classi. SVM è particolarmente efficace per dati ad alta dimensione e per problemi di classificazione binaria.

Entrambi i modelli sono stati testati su un dataset di esempio (ad esempio, il dataset di **Breast Cancer** fornito da **sklearn**). Per ogni modello sono stati calcolati diversi indicatori di performance, tra cui:

- **Accuracy**: percentuale di predizioni corrette.
- **Precision**: percentuale di predizioni positive corrette.
- **Recall**: percentuale di veri positivi su tutti i positivi reali.
- **F1-Score**: media armonica tra precisione e recall.



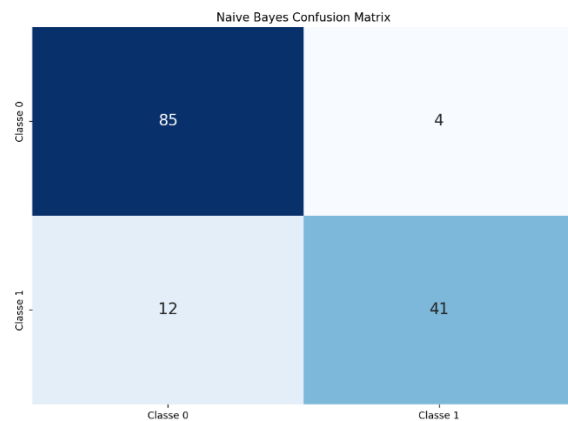
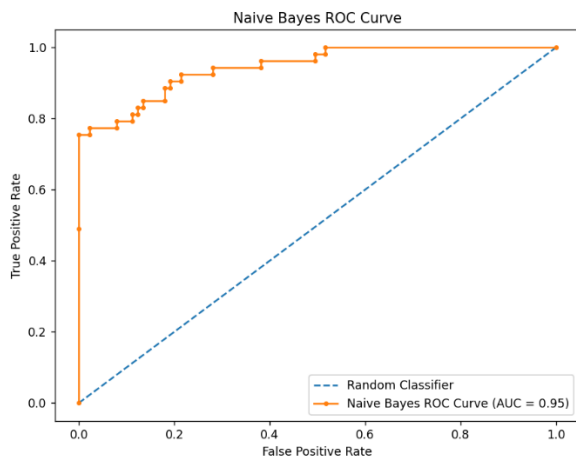
SPIEGAZIONE

SVM: La precisione inizia a un livello moderato (circa 0.80) con un set di addestramento piccolo (50 campioni) e migliora gradualmente all'aumentare della dimensione del set di addestramento. Raggiunge una precisione elevata quando il set di addestramento supera i 200 campioni.

Bayes: La precisione è inizialmente più bassa rispetto a SVM con un set di addestramento piccolo, ma migliora rapidamente. Tuttavia, dopo i 150-200 campioni, la precisione di Bayes viene superata da quella di SVM.

Inoltre, sono stati tracciati i grafici delle **curve ROC**, e le **matrici di confusione** per una comparazione visiva delle performance.

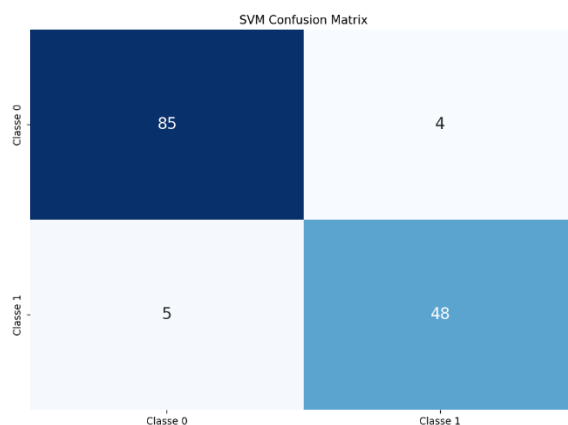
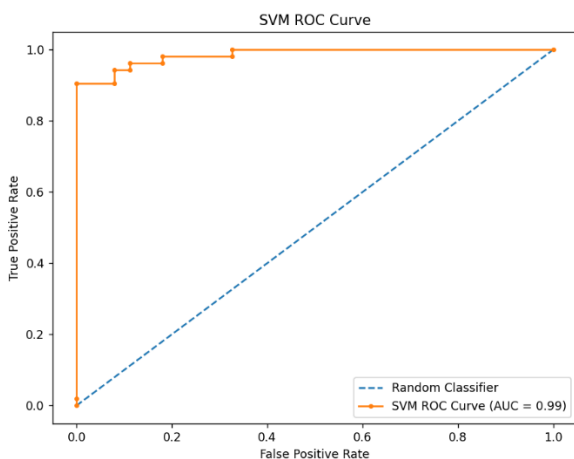
NAIVE BAYES:



SPIEGAZIONE

AUC = 0.95: L'area sotto la curva ROC è pari a **0.95** eccellente

SVM:



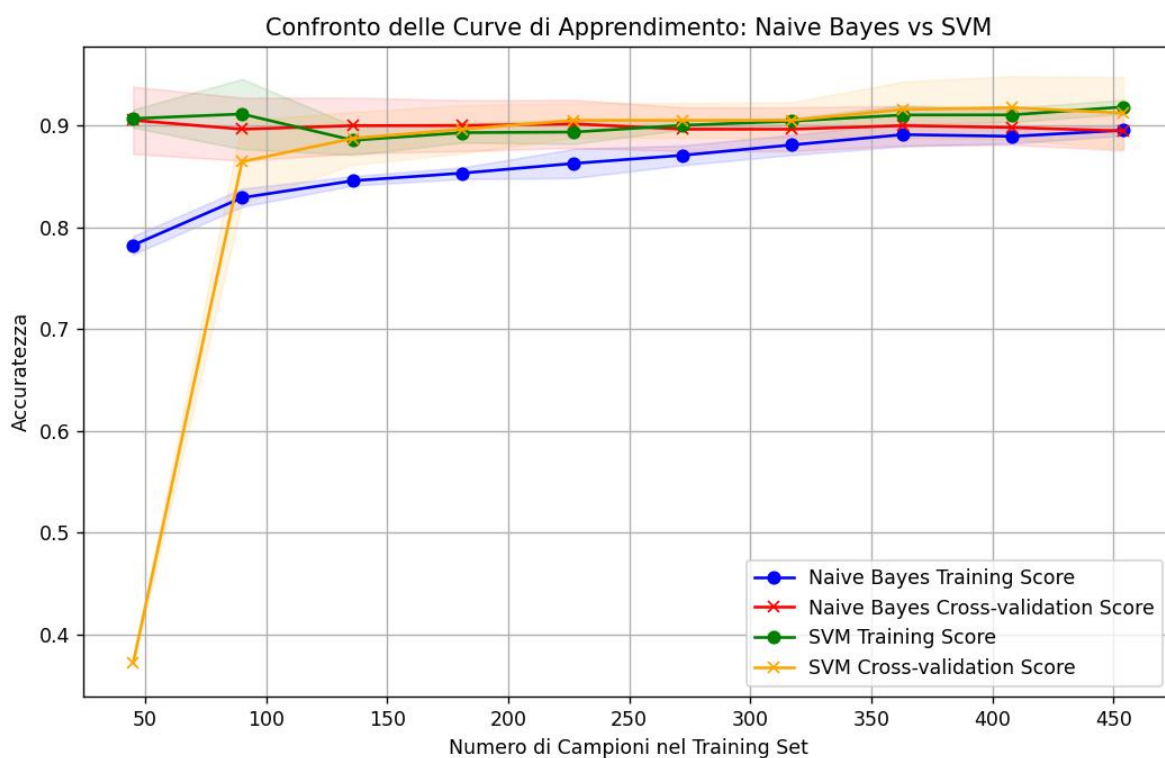
SPIEGAZIONE

AUC = 0.95: L'area sotto la curva ROC è pari a **0.99**, indicando un'ottima capacità discriminante del modello (valori vicini a 1 indicano alte prestazioni).

Il modello ha una buona performance, ma potrebbe essere migliorato ulteriormente riducendo il numero di falsi negativi (aumentando il recall).

Curves di Apprendimento

Le **curves di apprendimento** mostrano come cambia la performance del modello (sia sul training set che sul validation set) in relazione al numero di esempi nel training set. Le curve aiutano a capire se un modello sta soffrendo di overfitting o underfitting.



Naive Bayes

- **Training Score:** La curva del training score inizia alta e rimane relativamente stabile all'aumentare della dimensione del training set. Questo indica che il modello Naive Bayes si adatta bene ai dati di training.
- **Cross-validation Score:** La curva del cross-validation score inizia più bassa ma aumenta gradualmente, avvicinandosi al training score man mano che il training set cresce. Questo suggerisce che il modello generalizza bene sui dati non visti.
- **Convergenza delle curve:** Le curve di training e cross-validation convergono a un valore simile, indicando che il modello **Naive Bayes** non soffre di overfitting.

Support Vector Machine (SVM)

- **Training Score:** La curva del training score inizia alta e rimane stabile o aumenta leggermente all'aumentare della dimensione del training set. Questo indica che il modello SVM si adatta molto bene ai dati di training.
- **Cross-validation Score:** La curva del cross-validation score inizia più bassa ma aumenta rapidamente, avvicinandosi al training score man mano che il training set cresce. Questo suggerisce che il modello generalizza bene sui dati non visti.
- **Convergenza delle curve:** Le curve di training e cross-validation convergono a un valore simile, indicando che il modello **SVM** non soffre di overfitting.

APPENDIMENTO: CONCLUSIONI

Algoritmo	Accuratezza	Varianza	Deviazione	F1-score	AVG(P)	AUC
SVM	0.944	//	//	0.93	//	//
K-means	0.854	//	//	0.844	0.751	//
Random Forest	0.942	0.000	0.018	0.917	0.889	0.976
Decision Tree	0.871	0.001	0.019	0.831	0.743	0.868
Logistic Regression	0.982	0.000	0.0070	0.976	0.970	0.995
Multinomial Naive Bayes	0.887	//	//	0.88	//	//

Capitolo 5) Ragionamento logico e Prolog

Il ragionamento logico si differisce da quello probabilistico in quanto in questo caso il ragionamento avviene usando la logica matematica. Viene costruita una cosiddetta *Knowledge Base*, contenente gli assiomi (ovvero affermazioni sempre vere). Gli assiomi si dividono a loro volta in regole e fatti. I fatti sono verità immutabili, una regola è una dichiarazione che afferma che qualcosa è vero in base ad altre cose che sono vere.

Ho utilizzato la libreria *pyswip* per il *Prolog*. Utilizzando PySwip, è possibile eseguire query su questa KB, permettendo di ottenere informazioni specifiche in base a condizioni impostate

Prolog è un linguaggio di programmazione dichiarativo basato sul ragionamento logico. Nel mio caso sono stati definiti:

- **Fatto data:** rappresenta le informazioni estratte da un file CSV e trasformate in termini Prolog.
- **Regola query:** permette di interrogare il database per recuperare informazioni basate su condizioni specifiche

Regola generica:

data(ID, Diagnosis, Feature1, Feature2, Feature3, Feature4, Feature5, ..., Feature30).

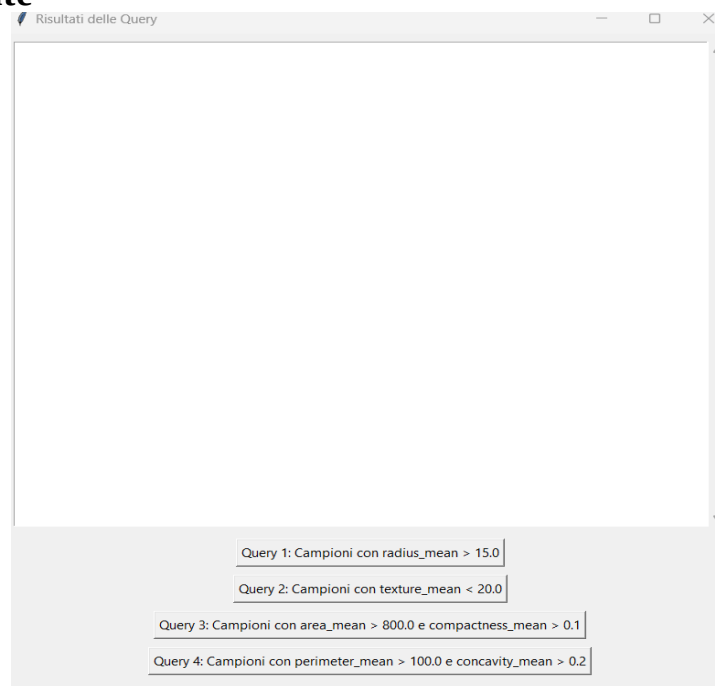
Ogni riga del dataset viene trasformata in un termine data(...) e scritta in un file .pl, che rappresenta la KB di Prolog.

È stata sviluppata una GUI con **Tkinter**, che permette di:

Visualizzare i risultati delle query in un'area di testo scorrevole.

Eseguire diverse interrogazioni sulla Knowledge Base con semplici pulsanti.

Query Implementate



Query Selezionata



Questo sistema mostra come combinare **ragionamento logico e apprendimento automatico**, utilizzando Prolog per gestire le informazioni strutturate e Tkinter per interagire con l'utente

Capitolo 6) Sistema di Prenotazione con CSP

Questo progetto implementa un sistema di prenotazione per esami medici utilizzando la programmazione a vincoli (**Constraint Satisfaction Problem - CSP**). Il sistema permette agli utenti di selezionare uno specialista, visualizzare le sue disponibilità e prenotare un appuntamento. Il modello CSP è costruito sulla libreria constraint, che gestisce le variabili, i domini e i vincoli associati agli specialisti.

Sono definiti tre elenchi principali:

- specialisti: una lista di medici con le loro specializzazioni.
- giorni_settimana: i giorni disponibili per le visite.
- turni_disponibili: le fasce orarie disponibili per ogni giorno.

```

specialisti = ["Dr. Rossi (Radiologo)", "Dr. Bianchi (Chirurgo)", "Dr. Verdi (Oncologo)"]
giorni_settimana = ["lunedì", "martedì", "mercoledì", "giovedì", "venerdì", "sabato"]
turni_disponibili = ["9:00-12:00", "12:00-15:00", "15:00-18:00"]

```

Implementazione della Classe MedicoCSP

```
class MedicoCSP(Problem):
    Qodo Gen: Options | Test this method
    def __init__(self, specialista):
        super().__init__()
        self.specialista = specialista
        self.addVariable("giorno", giorni_settimana)
        self.addVariable("orario", turni_disponibili)

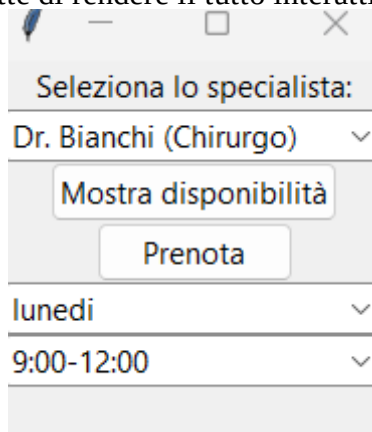
    Qodo Gen: Options | Test this method
    def get_disponibilita(self):
        soluzioni = self.getSolutions()
        giorni = sorted(set(s["giorno"] for s in soluzioni))
        orari = sorted(set(s["orario"] for s in soluzioni))
        return giorni, orari
```

Creazione delle Istanze di CSP

```
def crea_problemi():
    medici = [MedicoCSP(specialisti[i]) for i in range(3)]
    medici[0].addConstraint(lambda g, o: o in ["9:00-12:00", "12:00-15:00"] if g == "lunedì" else o in ["15:00-18:00"], ["giorno", "o"])
    medici[1].addConstraint(lambda g, o: o in ["9:00-12:00", "12:00-15:00"] if g == "martedì" else o in ["15:00-18:00"], ["giorno", "o"])
    medici[2].addConstraint(lambda g, o: o in ["9:00-12:00", "12:00-15:00"] if g == "giovedì" else o in ["15:00-18:00"], ["giorno", "o"])
    return medici
```

N.B: Consultare codice per visionare il restante codice

L'integrazione con **Tkinter** permette di rendere il tutto interattivo e accessibile all'utente.



Seleziona lo specialista:

Dr. Bianchi (Chirurgo) ▼

Mostra disponibilità

Prenota

lunedì ▼

9:00-12:00 ▼

Riferimenti bibliografici

Apprendimento supervisionato: D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. Cambridge University Press. 3rd Ed. [Ch.7]

Apprendimento non supervisionato: D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. Cambridge University Press. 3rd Ed. [Ch.10]